Advantages of Randomized QuickSort over Normal (Deterministic) Quicksort

Normal Quicksort

In normal QuickSort either we take first element as Pivot element or last element as pivot element.

Then we compare the pivot element with the remaining elements.

Element is marked if it is greater than the pivot element.

Then we swap the pivot element with the marked element.

For eg:-

Consider the following elements

5, 3, 2, 7, 8, 6

↑
pivot

Here we select '6' as pivot element. Then we compare the remaining elements with the pivot element.

First, We Compare $5 < 6$ ✓

Then $3 < 6$ ✓

$2 < 6$ ✓

$7 < 6$ ✗.

So. 7 is marked.

Then We swap pivot element (6) and the marked element (7)

The array looks like.

| 5 | 3 | 2 | 6 | 8 | 7 |

Then we partition the array as shown above, and we apply quicksort individually to both of these partitions.

ie; We apply Quicksort to the first partition 5, 3, 2 by taking '2' as pivot element and 8, 7, by taking 7 as

pivot element.

Worst Case

In quicksort worst case occurs when the given array is already sorted.

i.e. Consider the following example.

1 2 3 4 5

Here, We take the last element 5 as the pivot element.

Then we compare the pivot element with all other elements.

1, 2, 3, 4, 5
↑ pivot.

1 < 5 ✓
2 < 5 ✓
3 < 5 ✓
4 < 5 ✓

Here the partition will be like 1 2 3 4 | 5

In the next iteration, the last element 4 will be taken as pivot element, and it is compared with all other elements.

1, 2, 3, 4 | 5

↑ pivot

1 < 4

2 < 4

3 < 4

Partition will be 1, 2, 3 | 4 | 5

⇒ So each time the partition will be (n-1),

⇒ We can represent this using recurrence equation

$$T(n) = T(n-1) + n$$

Solving this equation, we will get complexity as $O(n^2)$

So, the Worst case Complexity of quicksort will be $O(n^2)$.

Randomized QuickSort

We use Randomized Quicksort in order to solve or bring down the worst case complexity of deterministic quick sort algorithm.

In Randomized Quicksort, we select the pivot element as random.

Because the pivot element is randomly chosen, we expect the split of the input array to be reasonably well balanced.

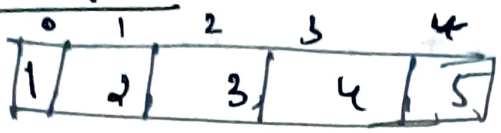
Algorithm

Randomized Quicksort (A, p, r)

if p < r

q

For eg:-



Here we select i = Random Function Generator, a random number.

Consider i = 2;

Then we exchange the last element with the element in 2nd position.

Resultant array will be like

1, 2, 5, 4, 3

Then we apply normal Quick Sort algorithm to this.

i.e. we take last element as pivot element, and Compare all the other elements with

pivot element.

1, 2, 5, 4, 3

↑ pivot.

1 < 3 ✓

2 < 3 ✓

5 < 3 ✗

Mark element 5, then swap 5, and 3

Resultant array will look like.

1, 2, |3| 4, 5

⇒ It will divide the array into two equal halves. o there then (n-1).

⇒ Here the recurrence equation will be $T(n) = 2T(n/2) + b$.

⇒ The Complexity of this recurrence equation will be.

$T(n) = O(n \log n)$.

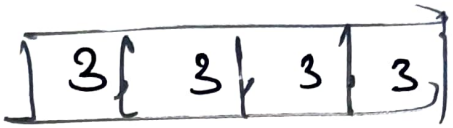
As we can see, here we bring down the worst case complexity of deterministic quicksort $O(n^2)$ into $O(n \log n)$ in Randomized Quick sort.

Algm.

```
Randomized QuickSort (A, p, r).
    if p<r
        q ← Randomized Partition (A, p, r)
            Randomized Quicksort (A, p, q-1)
            Randomized QuickSort (A, q, r)
```

```
Randomized Partition (A, p, r).
    i ← Random Function Generation (p, r)
        Exchange A[r] ⟷ A[i].
        Partition (A, p, r).
```

Randomized Quicksort will produce the worst case complexity $O(n^2)$ only when the array contains same elements.

For eg: 

All other cases the complexity will be less than $O(n^2)$